Advanced Database Systems Portfolio Summary

Ezenduka Chukwubuikem | Student ID: 301913360
MIT 8103 - 2026/2027 First Semester

Repository: https://github.com/Raphael632/advanced-database-portfolio
Selected organisation: National Bureau of Statistics (Nigeria).

Case study and data

The Nigeria Food Price Observatory is a simulated research and data-quality system using the National Bureau of Statistics (NBS) Selected Food Price Watch [1]. Analysts compare published food prices, data stewards review provenance and labels, and an administrator manages imports. The 2025 snapshot contains **3,528 observations, 12 months, seven geographies and 43 distinct source labels**. National figures are imported directly, not calculated as an average of zones. Fictional workflow records are explicitly labelled; source prices are not fabricated.

Portfolio 1 - Design and modelling

Eight strict SQLite tables separate publishers, resources, publications, items, geographies, observations, demo users and review tasks. The observation grain is publication-item-geography. Foreign keys, unique constraints and positive integer-kobo checks protect this structure; triggers prevent price updates/deletes and require review-version increments. Normalisation, an ER diagram and source-cell provenance are documented. The two smoked-fish labels remain distinct because equivalence is not established. A fresh rebuild matched every row in all eight original tables.

Portfolio 2 - Query processing and optimisation

Eight SQL analyses produced 106 independently checked result rows using joins, aggregation, ranking and calendar-month comparisons. Two covering indexes and removal of an unnecessary text conversion were evaluated with actual execution plans [2]. The captured experiment used 31 alternating rounds of 200 executions per side, after warm-up; all ordered before/after results matched.

Measured case	Before / after median	Recorded speed ratio
Series lookup	107.612 / 95.395 microseconds	1.13x
Top-price lookup	189.969 / 135.632 microseconds	1.40x
Predicate rewrite	203.587 / 95.852 microseconds	2.12x
Full-coverage control	1,677.869 / 1,695.345 microseconds	0.99x - no useful gain

These are the original 3 October measurements, not production throughput claims. Indexes added 128 KiB. The fresh audit reproduced query results and similar timing patterns; its rewrite ratio was 1.99x. Both runs are retained separately.

Portfolio 3 - Transactions and concurrency

Nine controlled, two-connection scenarios verify commit, complete rollback, savepoints, WAL reader snapshots and writer contention [3]. A deliberately stale form overwrites a note when the application ignores its loaded version. The corrected update includes a version predicate, rejects a stale edit through a zero-row result and requires refresh/reconciliation. The tests distinguish this application-level lost edit from SQLite isolation. Error paths roll back transactions; normal reopen checks are not presented as crash-recovery tests.

Reviewer sequence	Observed state
A and B load the task	Both see version 0
A saves WHERE version = 0	One row saved; version becomes 1
B submits its old version 0	Zero rows; A's note remains intact
B refreshes and reconciles	New accepted edit produces version 2

Portfolio 4 - NoSQL model

TinyDB 4.9.0 stores **516 monthly-item documents** with embedded regional observations and publication provenance. Closing and reopening storage preserved all 3,528 source rows; six document queries matched independent SQL references. Fictional review documents demonstrate insert/read/update/delete operations and optional checklists. Validation rejects duplicate identities and invalid values. TinyDB suits this small single-writer demonstration, but does not supply the production concurrency, native indexes or ACID guarantees needed for shared reviewer writes [4].

Portfolio 5 - Distributed/cloud exercise

Separate local SQLite files demonstrate asynchronous review-event replication, applied-sequence checkpoints, freshness-aware routing, recovery and month-range shards. Eight scenarios show labelled stale reads, refusal of unsafe operations during a simulated outage, atomic replay recovery and one acknowledged edit absent from an older backup. Each shard holds 1,764 observations; their exact union reconstructs all 3,528 rows. A missing shard prevents a supposedly complete global result.

A proposed managed PostgreSQL primary, standby, read replica and retained backups is documented with availability/consistency tradeoffs [5]. **No cloud cluster was deployed.** Paused workers and availability flags simulate faults locally; the exercise does not measure network partitions, automatic failover, production RPO/RTO or uptime.

Cross-portfolio verification

On 5 October, **54 unit tests passed** (20 + 12 + 12 + 10), alongside nine Portfolio 3 and eight Portfolio 5 integration scenarios. All five phases were re-executed. Source hashes remained unchanged; local document/image links resolved. Reproducible scripts, dated outputs, figures and audit logs are included in the public repository. This verification covers defined cases, not every possible input or deployment.

Technical conclusions and limitations

The case study shows why data provenance, transaction boundaries and access patterns should shape database design. Targeted indexes improved tested lookups but did not improve full-coverage aggregation. Version checking protects a reviewer's stale form, while document embedding simplifies monthly retrieval at the cost of repeated metadata. Replicas and backups can both lag acknowledged work, making freshness requirements and recovery points explicit design decisions.

NBS downloads are publicly accessible, but unrestricted redistribution was not established. Raw workbooks and populated databases are therefore excluded; source URLs/checksums and reconstruction instructions are provided.

AI-use declaration

AI tool used: OpenAI Codex. I used OpenAI Codex to assist with source research, SQL/Python code generation and revision, explanations, diagrams, document preparation and automated test execution. I directed the phased project workflow and requested corrections to GitHub organisation and diagram display. On 18 October 2026, I confirmed that I had reviewed Portfolios 1-5 and found them satisfactory. I first reviewed and approved Portfolio 1 on 14 October 2026. My review and approval are recorded separately from the automated execution results in the test and audit evidence. Python, SQLite, TinyDB, openpyxl, Git and ReportLab are implementation tools/libraries, not additional AI services.

Individual reflection

Through reviewing this project, I learned that Nigerian food-price data was a well-suited dataset for the portfolio. It provided a relevant Nigerian context for the database work across the five phases. I found Portfolio 3, which focuses on transactions and concurrency, somewhat challenging. I have reviewed all five portfolios and am satisfied with the completed work.

Selected references

[1] NBS. Selected Food Price Watch, 2025 resources; source manifest in Portfolio 1. Source.

[2] SQLite. Query Planning; measured plans and SQL are in Portfolio 2. Source.

[3] SQLite. Isolation in SQLite; transaction documentation cited in Portfolio 3. Source.

[4] TinyDB. Introduction and suitability limits; usage sources in Portfolio 4. Source.

[5] PostgreSQL. Log-Shipping Standby Servers; proposed architecture only. Source.

[6] SQLite. Backup API; implemented local recovery exercise in Portfolio 5. Source.

Sources consulted during 1-5 October 2026. Full URLs, source checksums, implementation details and evidence are available in the phase reports and manifests.